

DÉVELOPPEMENT DE MODULES DRUPAL AVANCÉS

Rapport de Formation Technique (5 Jours)

Réalisé par

YAHYA EL OMARI

Encadré par

ABDELMAJID BENDRIF & HAMZA BAHLAOUANE

16 mars 2026

JOUR 1 : CONCEPTS AVANCÉS, ENTITÉS ET API FIELD

Cette journée fut consacrée à l'exploration des API avancées de Drupal, notamment l'altération des entités existantes, la gestion fine des accès et la génération de code via Drush.

1. Altération de l'Entité User : Ajout d'une Contrainte

Drupal permet de modifier la définition des champs de base via le hook `hook_entity_base_field_info_alter()`. Nous l'avons utilisé pour injecter une contrainte de validation personnalisée sur le champ mot de passe (`pass`) de l'utilisateur.

Listing 1 – Altering User Entity

```
/**
 * Implements hook_entity_base_field_info_alter().
 */
function user_advanced_validation_entity_base_field_info_alter(&$fields,
  EntityTypeInterface $entity_type) {
  if ($entity_type->id() === 'user' && isset($fields['pass'])) {
    // On ajoute la contrainte personnalisée au champ mot de passe
    $fields['pass']->addConstraint('PasswordPolicyConstraint', []);
  }
}
```

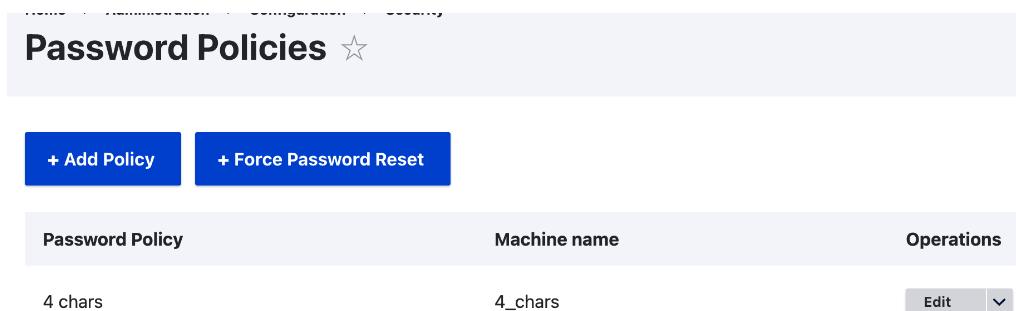


FIGURE 1 – Interface de configuration des politiques de mot de passe dans Drupal

La contrainte `PasswordPolicyConstraint` utilise ensuite le service `password_policy.validator` (du module *Password Policy*) via l'injection de dépendances pour vérifier la conformité du mot de passe.

2. Qu'est-ce que AccessResult et comment ça fonctionne ?

`AccessResult` est l'objet utilisé par Drupal pour déterminer si un utilisateur peut effectuer une action (voir, éditer, supprimer). Contrairement à un simple booléen, il transporte le contexte de la décision.

Il existe trois états principaux :

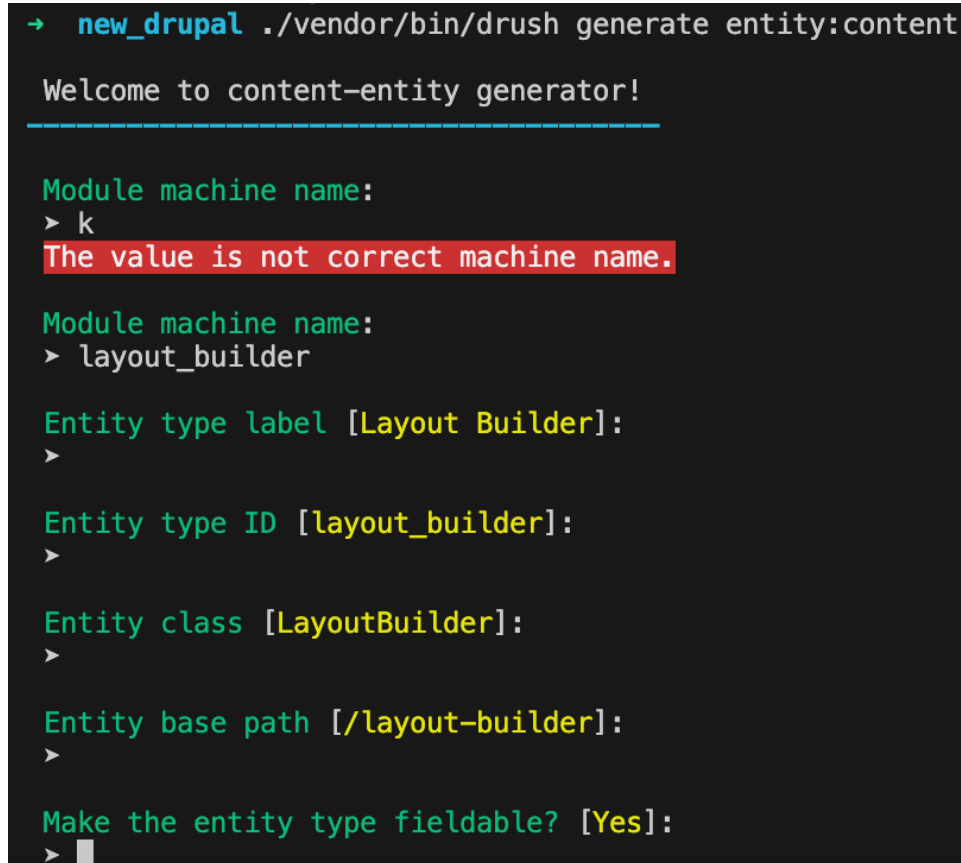
- Allowed : Accès autorisé.
- Forbidden : Accès explicitement refusé (l'emporte sur tout).
- Neutral : Ne se prononce pas (laisse d'autres modules décider).

L'objet `AccessResult` doit souvent être accompagné de méthodes comme `addCacheContexts()` pour garantir que la décision d'accès est correctement mise en cache selon l'utilisateur ou ses permissions.

3. Génération Rapide de Boilerplate (Scaffolding)

Pour éviter d'écrire manuellement tout le code répétitif d'une entité personnalisée, nous utilisons la commande **drush generate** :

```
./vendor/bin/drush generate entity:content
```



```
→ new_drupal ./vendor/bin/drush generate entity:content

Welcome to content-entity generator!
=====

Module machine name:
> k
The value is not correct machine name.

Module machine name:
> layout_builder

Entity type label [Layout Builder]:
>

Entity type ID [layout_builder]:
>

Entity class [LayoutBuilder]:
>

Entity base path [/layout-builder]:
>

Make the entity type fieldable? [Yes]:
>
```

FIGURE 2 – Utilisation de drush generate pour le scaffolding d'une entité personnalisée

Cela génère automatiquement tous les fichiers nécessaires : l'interface, la classe d'entité, le gestionnaire d'accès et les formulaires.

4. Obtenir la Définition d'un Champ par le Code

Selon le type de champ (base ou configurable), on utilise l'un des deux services suivants :

- Champ de base (ex : title, created) :

```
$definition = \Drupal::service('entity_field.manager')->getBaseFieldDefinitions('node');
```

- Champ ajouté via l'UI (Field Config) :

```
$field_config = FieldConfig::loadByName('node', 'article', 'field_image');
```

5. Plusieurs Formateurs pour un seul Type de Champ ?

Oui, c'est tout à fait possible. Un *Field Formatter* est une annotation **@FieldFormatter**. Un même **field_types** peut être utilisé par des dizaines de formateurs différents. Cela permet à l'administrateur de choisir entre le rendu par défaut et le vôtre dans l'onglet "Manage Display".

6. Récupérer la Configuration d'un Module via Drush

Pour inspecter les réglages enregistrés pour un module :

- Lister les clés de config : `drush config:list | grep nom_du_module`
- Lire une configuration spécifique : `drush config:get nom_du_module.settings`

```
→ new_drupal ./vendor/bin/drush config:get user.settings
_core:
  default_config_hash: Y-Xpk050V3opj3WiFb4oUbVT9S7fDA1XLpig0-qVMVQ
langcode: en
anonymous: Anonymous
verify_mail: true
notify:
  cancel_confirm: true
  password_reset: true
  status_activated: true
  status_blocked: false
  status_canceled: false
  register_admin_created: true
  register_no_approval_required: true
  register_pending_approval: true
register: admin_only
cancel_method: user_cancel_block
password_reset_timeout: 86400
password_strength: true
```

FIGURE 3 – Lecture de la configuration utilisateur via Drush

JOUR 2 : CYCLE DE VIE, BASE FIELDS ET THEME HOOKS

Cette journée a porté sur la manipulation de la structure des données au niveau profond et sur la compréhension du cycle de vie des entités et des thèmes dans Drupal.

1. Ajouter un champ de base (**hook_entity_base_field_info**)

Ce hook permet de définir des champs appartenant à la structure intrinsèque de l'entité. Contrairement aux champs configurables via l'interface, ceux-ci sont définis dans le code.

Listing 2 – Adding a base field to Nodes

```
function user_advanced_validation_entity_base_field_info(
  EntityTypeInterface $entity_type) {
  if ($entity_type->id() === 'node') {
    $fields['custom_reference'] = BaseFieldDefinition::create('string')
      ->setLabel(t('Référence personnalisée'))
      ->setSettings(['max_length' => 50])
      ->setDisplayOptions('view', ['label' => 'above', 'type' => 'string'])
      ->setDisplayOptions('form', ['type' => 'string_textfield']);
    return $fields;
  }
}
```

Référence personnalisée

Un code de suivi interne.

FIGURE 4 – Affichage du champ de base personnalisé dans le formulaire d'édition de contenu

2. Rôles des hooks d'installation et de mise à jour

- `hook_install()` : S'exécute une seule fois lors de l'activation initiale du module. Idéal pour initialiser des données par défaut.
- `hook_update_n()` : Système de migration de base de données. Il permet d'appliquer des changements de structure (ex : nouvelles colonnes) sur un site déjà installé. La convention de nommage respecte la version de Drupal et l'ordre séquentiel.

3. Manipulation avant sauvegarde (**hook_ENTITY_TYPE_presave**)

Ce hook intervient juste avant l'écriture en base de données. Nous l'avons utilisé pour préfixer automatiquement les titres des articles par "HEY- ".

Listing 3 – Prefixing node titles

```
function mon_module_node_presave(EntityInterface $node) {
  if ($node->bundle() === 'article') {
    $current_title = $node->getTitle();
```

```

if (strpos($current_title, 'HEY-') !== 0) {
    $node->setTitle('HEY-' . $current_title);
}
}
}

```

✓ Article HEY- prefixTest has been created. ✕

View	Edit	Delete	Revisions	Devel
-------------	------	--------	-----------	-------

[Home](#)

HEY- prefixTest ☆

FIGURE 5 – Résultat du hook presave : préfixe automatique du titre après sauvegarde

Note sur **\$entity->original** : Disponible dans les hooks de sauvegarde, cet objet contient l'état de l'entité *avant* les modifications actuelles, permettant de comparer les valeurs (ex : vérifier si un statut a changé).

4. Surcharge de Thème et Suggestions

Drupal offre une grande flexibilité pour modifier le rendu HTML :

- Overrider un Theme Hook : Via **hook_theme_registry_alter()**, on peut remplacer le template d'un autre module par le sien.
- Suggestions de Thème : **hook_theme_suggestions_alter()** permet d'ajouter des noms de fichiers templates possibles.

Listing 4 – Custom user theme suggestion

```

function mon_module_theme_suggestions_alter(array &$suggestions, array
    $variables, $hook) {
    if ($hook === 'user' && isset($variables['elements']['#view_mode'])) {
        $view_mode = $variables['elements']['#view_mode'];
        $suggestions[] = 'user__' . $view_mode;
    }
}

```

JOUR 3 : ARCHITECTURE MULTI-ENVIRONNEMENTS ET CONFIGURATION

L'objectif de cette journée était de mettre en place une architecture serveur isolée et de maîtriser la synchronisation de configuration entre différents environnements.

1. Mise en Place de l'Architecture Serveur

Description : Nous avons isolé deux environnements distincts dédiés au développement sous Drupal. L'objectif est de reproduire un cycle de vie d'application classique en instanciant un environnement de Développement (**dev-drupal.local**) et un environnement de Production (**prod-drupal.local**).

A. Création de l'Arborescence

Les répertoires suivants ont été instanciés pour accueillir le code source :

- `/Users/mac/Documents/drupal/drupal_dev/web`
- `/Users/mac/Documents/drupal/drupal_prod/web`

B. Résolution DNS Locale (`/etc/hosts`)

Les entrées suivantes ont été ajoutées pour forcer la résolution vers la boucle locale (**127.0.0.1**) :

- `127.0.0.1 dev-drupal.local`
- `127.0.0.1 prod-drupal.local`

C. Configuration des Hôtes Virtuels Apache

Le serveur web Apache a été configuré via le fichier **httpd-vhosts.conf** pour traiter les requêtes entrantes par nom de domaine (*Name-based Virtual Hosting*).

2. Environnement de Développement

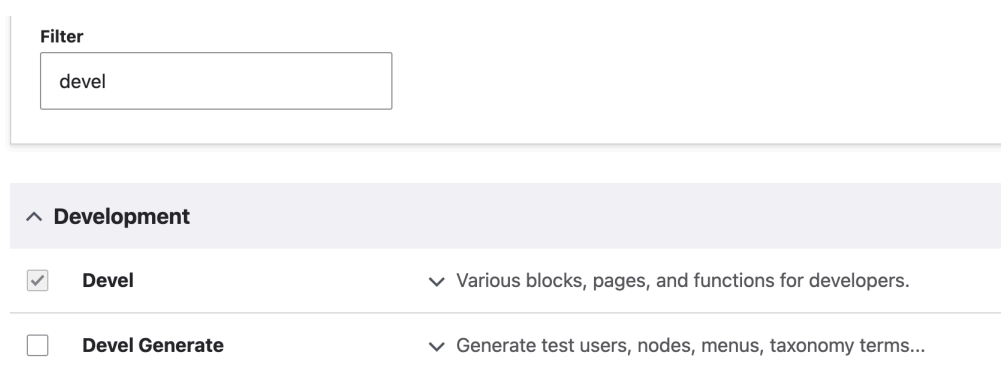


FIGURE 6 – Installation et activation de Devel dans l'environnement de développement

Dans l'environnement de développement, le module Devel a été installé via Composer et activé pour faciliter le développement.

Ensuite, une taxonomie et un type de contenu de base ont été créés pour tester la structure. Un rôle avec des permissions spécifiques a également été configuré.

FIGURE 7 – Création de contenu, taxonomie et configuration des permissions

3. Synchronisation de la Configuration (Full Sync)

L'objectif initial était d'exporter la configuration complète de l'environnement de développement pour l'importer en production afin de synchroniser les structures.

A. Tentative initiale et Échec (3 Erreurs)

Lors de la première tentative d'importation en production via l'interface d'administration (*Configuration Sync*), Drupal a bloqué le processus en affichant trois erreurs critiques empêchant toute modification.

```
In ConfigImportCommands.php line 291:
The import failed due to the following reasons:
Site UUID in source storage does not match the target storage.
Entities exist of type <em class="placeholder">Shortcut link</em> and <em class="placeholder">Shortcut set</em> <em class="placeholder">Default</em>.
These entities need to be deleted before importing.
Unable to install the <em class="placeholder">devel</em> module since it does not exist.

In ConfigImporter.php line 847:
There were errors validating the config synchronization.
Site UUID in source storage does not match the target storage.
Entities exist of type <em class="placeholder">Shortcut link</em> and <em class="placeholder">Shortcut set</em> <em class="placeholder">Default</em>.
These entities need to be deleted before importing.
Unable to install the <em class="placeholder">devel</em> module since it does not exist.
```

FIGURE 8 – Échec critique : 3 erreurs bloquantes (UUID, Devel, Raccourcis)

L'analyse de cet écran révèle :

- Une erreur d'UUID (Site ID mismatch).
- L'absence du module Devel sur le système de fichiers de production.
- Des conflits avec les entités de type *Shortcut*.

B. Étape 1 : Résolution de l'UUID (Passage à 2 Erreurs)

Pour que Drupal accepte l'importation d'un autre site, les deux instances doivent partager le même identifiant unique. Nous avons récupéré l'UUID en développement pour l'appliquer en production via Drush.

```
→ drupal_dev ./vendor/bin/drush cget system.site uuid
'system.site:uuid': 2042d66c-df5b-462e-a2e2-31559b60ce0b
```

FIGURE 9 – Récupération de l'UUID du site de développement


```

+ drupal_prod ./vendor/bin/drush cset system.site uuid 2042d66c-df5b-462e-a2e2-31559b60ce0b -y
// Do you want to update uuid key in system.site config?: yes.

```

FIGURE 10 – Forçage de l'UUID sur le site de production

Après avoir rafraîchi la page de synchronisation, l'erreur de "Site ID" disparaît, ne laissant plus que deux erreurs.

```

In ConfigImportCommands.php line 291:
The import failed due to the following reasons:
Entities exist of type <em class="placeholder">Shortcut link</em> and <em class="placeholder">Shortcut set</em> <em class="placeholder">Default</em>. These entities need to be deleted before importing.
Unable to install the <em class="placeholder">devel</em> module since it does not exist.

In ConfigImporter.php line 847:
There were errors validating the config synchronization.
Entities exist of type <em class="placeholder">Shortcut link</em> and <em class="placeholder">Shortcut set</em> <em class="placeholder">Default</em>. These entities need to be deleted before importing.
Unable to install the <em class="placeholder">devel</em> module since it does not exist.

```

FIGURE 11 – Progression : L'erreur d'UUID est résolue, il reste 2 blocages

C. Étape 2 : Nettoyage des Raccourcis (Passage à 1 Erreur)

Les entités de contenu par défaut (Default Shortcuts) créées lors de l'installation de la production entraînent en conflit avec les définitions exportées du développement.

```

+ drupal_prod ./vendor/bin/drush eval "print_r(\Drupal::entityTypeManager()->getStorage('shortcut')->loadMultiple());"
Array
(
    [1] => Drupal\shortcut\Entity\Shortcut Object
        (
            [entityTypeId:protected] => shortcut
            [enforceIsNew:protected] =>
            [typedData:protected] =>
            [originalEntity:protected] =>
            [cacheContexts:protected] => Array
                (
                )
            [cacheTags:protected] => Array
                (
                )
            [cacheMaxAge:protected] => -1
            [_serviceIds:protected] => Array
                (
                )
            [_entityStorages:protected] => Array
                (
                )
            [values:protected] => Array
                (
                    [id] => Array
                        (
                            [x-default] => 1
                        )
                )
        )
)

```

FIGURE 12 – Identification des raccourcis responsables du conflit

```

+ drupal_prod ./vendor/bin/drush eval "\$entities = \Drupal::entityTypeManager()->getStorage('shortcut')->loadMultiple(); foreach(\$entities as \$entity) { echo 'Deleting Shortcut: ' . \$entity->label() . PHP_EOL; \$entity->delete(); }"
Deleting Shortcut: Add content
Deleting Shortcut: All content

```

FIGURE 13 – Suppression manuelle des entités pour libérer le chemin

Une fois les raccourcis supprimés, l'interface n'affiche plus qu'un seul avertissement concernant le module Devel manquant.

```

In ConfigImportCommands.php line 291:
  The import failed due to the following reasons:
  Unable to install the <em class="placeholder">devel</em> module since it does not exist.

In ConfigImporter.php line 847:
  There were errors validating the config synchronization.
  Unable to install the <em class="placeholder">devel</em> module since it does not exist.

```

FIGURE 14 – Progression : Plus qu'une seule erreur (Module Devel)

D. Étape 3 : Résolution du module Devel et Import Final

Pour lever le dernier verrou, nous avons dû aligner la liste des modules. Bien que nous ne souhaitions pas Devel en production à terme, nous l'avons activé temporairement pour valider la synchronisation.

```

→ drupal_prod composer require drupal/devel
./composer.json has been updated
Running composer update drupal/devel
Loading composer repositories with package information
Updating dependencies
Lock file operations: 5 installs, 0 updates, 0 removals
  - Locking doctrine/common (3.5.0)
  - Locking doctrine/event-manager (2.1.1)
  - Locking doctrine/persistence (4.1.1)
  - Locking drupal/devel (5.5.0)
  - Locking psr/cache (3.0.0)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 5 installs, 0 updates, 0 removals
  - Installing psr/cache (3.0.0): Extracting archive
  - Installing doctrine/event-manager (2.1.1): Extracting archive
  - Installing doctrine/persistence (4.1.1): Extracting archive
  - Installing doctrine/common (3.5.0): Extracting archive
  - Installing drupal/devel (5.5.0): Extracting archive
Package pear/console_getopt is abandoned, you should avoid using it
Generating autoload files
47 packages you are using are looking for funding.
Use the `composer fund` command to find out more!
No security vulnerability advisories found.
Using version ^5.5 for drupal/devel

```

FIGURE 15 – Alignement temporaire des modules pour l'importation

```

[notice] Synchronized configuration: create views.view.block_content.
[notice] Synchronized configuration: create views.view.comment.
[notice] Synchronized configuration: create views.view.comments_recent.
[notice] Synchronized configuration: create views.view.content.
[notice] Synchronized configuration: create views.view.content_recent.
[notice] Synchronized configuration: create views.view.files.
[notice] Synchronized configuration: create views.view.frontpage.
[notice] Synchronized configuration: create views.view.glossary.
[notice] Synchronized configuration: create views.view.taxonomy_term.
[notice] Synchronized configuration: create views.view.user_admin_people.
[notice] Synchronized configuration: create views.view.watchdog.
[notice] Synchronized configuration: create views.view.who_s_new.
[notice] Synchronized configuration: create views.view.who_s_online.
[notice] Synchronized configuration: update system.site.
[notice] Finalizing configuration synchronization.
[success] The configuration was imported successfully.
→ drupal_prod ./vendor/bin/drush cr
[success] Cache rebuild complete.

```

FIGURE 16 – Succès final : Synchronisation complète effectuée

4. Solutions de Déploiement Granulaire (Split & Features)

A. Optimisation via Config Split

Après avoir validé que la synchronisation fonctionnait, nous avons nettoyé l'environnement de production pour mettre en place une solution plus pérenne.

```

→ drupal_prod ./vendor/bin/drush pmu devel -y
[success] Successfully uninstalled: devel
→ drupal_prod composer remove drupal/devel
./composer.json has been updated
Running composer update drupal/devel
Loading composer repositories with package information
Updating dependencies
Lock file operations: 0 installs, 0 updates, 5 removals
- Removing doctrine/common (3.5.0)
- Removing doctrine/event-manager (2.1.1)
- Removing doctrine/persistence (4.1.1)
- Removing drupal/devel (5.5.0)
- Removing psr/cache (3.0.0)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 0 installs, 0 updates, 5 removals
- Removing psr/cache (3.0.0)
- Removing drupal/devel (5.5.0)
- Removing doctrine/persistence (4.1.1)
- Removing doctrine/event-manager (2.1.1)
- Removing doctrine/common (3.5.0)
0/5 [=====] %Deleting /Users/mac/Documents/drupal/drupal_prod/web/modules/contrib/devel - deleted
Package pear/console_getopt is abandoned, you should avoid using it. No replacement was suggested.
Generating autoload files
44 packages you are using are looking for funding.
Use the 'composer fund' command to find out more!
No security vulnerability advisories found.
→ drupal_prod rm -rf config/sync/*
zsh: sure you want to delete more than 100 files in /Users/mac/Documents/drupal/drupal_prod/config/sync [yn]? y

```

FIGURE 17 – Nettoyage de l'environnement de production avant configuration du Split

Le module Config Split a ensuite été configuré pour exclure dynamiquement Devel de l'exportation vers la production.

Modules

- ☐ Database Logging
- ☐ Datetime
- ☐ Datetime Range
- ☒ Devel
- ☒ Devel Generate
- ☐ Field
- ☐ Field Layout

Select modules to split. Configuration depending on the modu

Configuration items

- ☐ core.menu.static_menu_link_overrides
- ☐ dblog.settings
- ☒ devel.settings
- ☒ devel.toolbar.settings
- ☐ editor.editor.basic_html
- ☐ editor.editor.full_html
- ☐ field.field.block_content.basic.body

Select configuration to split. Configuration depending on split

FIGURE 18 – Configuration du Split pour ignorer Devel en production

Nous avons configuré le split dans les fichiers **settings.php** pour qu'il soit actif en développement (**TRUE**) et inactif en production (**FALSE**). Cela permet d'avoir des pipelines de déploiement propres où les outils de debug ne polluent pas les environnements finaux.

B. Utilisation de Features pour le déploiement ciblé

Pour un déploiement plus granulaire et modulaire, nous avons utilisé le module Features. Contrairement au *Full Sync*, Features permet d'isoler des fonctionnalités précises (ex : un type de contenu et ses champs) dans un module exportable.

```
drupal_dev ./vendor/bin/drush features-list
```

Name	Machine name	Status	Version	State
Default comments	comment	Not exported		
Article	article	Not exported		
featuresContentTypeTest	featurescontenttypetest	Not exported		
Basic page	page	Not exported		
project content type name	project_content_type_name	Not exported		
User	user	Not exported		Changed
Core	core	Not exported		Changed
Site	site	Not exported		

FIGURE 19 – Liste des composants disponibles pour l'export via Drush Features

1. Création et Exportation (Côté Développement)

Une *feature* a été créée pour regrouper des éléments spécifiques. Cette étape génère un module physiquement présent dans le dossier `modules/custom`.

^ Content type (3)

☐ Article (article)
☐ Basic page (page)
☐ project content type name (project_content_type_name)

☒ featuresContentTypeTest (featurescontenttypetest)

FIGURE 20 – Sélection des composants et génération du module de fonctionnalité

2. Activation et Importation (Côté Production)

Une fois le module transféré et activé en production, la configuration est immédiatement appliquée.

featuresContentTypeTest	Manage fields
project content type name	Manage fields

FIGURE 21 – État de la configuration en production après activation du module

3. Test de Conflit et Source de Vérité

Nous avons simulé une modification manuelle en production (changement de nom) pour vérifier la robustesse du système.

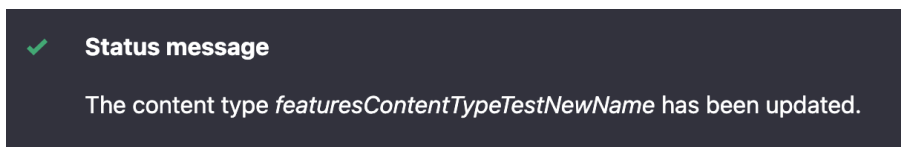


FIGURE 22 – Modification manuelle effectuée en production (*Overridden*)

Ensuite, nous avons ajouté un nouveau champ en développement et mis à jour la feature.

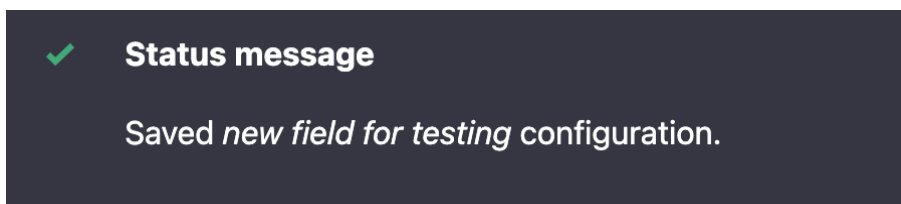


FIGURE 23 – Ajout d'un nouveau champ **field_test** en développement

L'importation en production a permis de constater que la modification manuelle est détectée comme une différence (*Diff*) et peut être écrasée pour revenir à l'état défini dans le code.

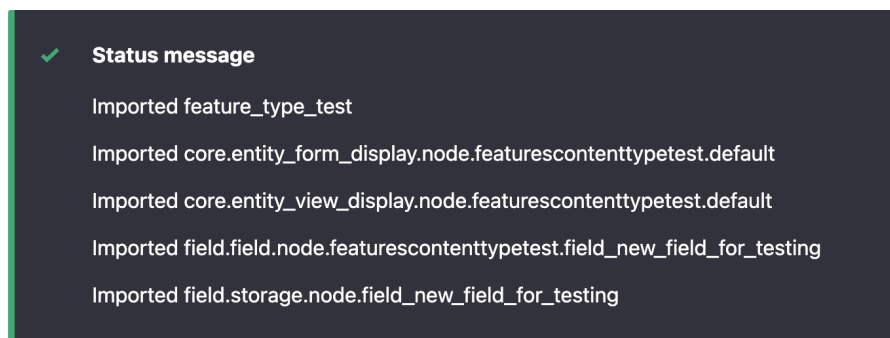


FIGURE 24 – Visualisation des différences entre le code et la base de données en production

5. Conclusion et Q&A

Question : *Le module Devel a-t-il été installé en production ?*

Réponse : Non. Grâce à Config Split, nous avons pu garder Devel en développement tout en garantissant qu'il soit ignoré lors de l'importation en production.

Question : *Que se passe-t-il si on modifie la configuration en production manuellement ?*

Réponse : Lors de l'importation (Sync ou Feature), Drupal considère le code comme la source de vérité. Les modifications manuelles en production seront écrasées par la configuration importée.

JOUR 4 : GESTION DU CACHE

L'objectif de cette journée était de se familiariser avec les mécanismes de mise en cache de Drupal à travers la création d'un module API minimaliste.

1. Utilisation du **max-age** (Invalidation Temporelle)

Nous avons forcé un cache temporel simple en utilisant un en-tête **Cache-Control**. Les données ne sont rafraîchies qu'après l'expiration du délai défini (TTL).

Listing 5 – Contrôleur utilisant max-age

```
return new JsonResponse($data, 200, ['Cache-Control' => 'max-age=3600']);
```

2. Utilisation des *Cache Tags* (Invalidation Événementielle)

La méthode recommandée utilise l'API de cache native de Drupal. En associant des *tags* de cache aux entités, Drupal invalide le cache de manière intelligente dès que le contenu est modifié.

Listing 6 – Utilisation des Cache Tags

```
$cache_metadata = new CacheableMetadata();  
$cache_metadata->addCacheTags(['node_list']);  
// ...  
$response = new CacheableJsonResponse($data);  
$response->addCacheableDependency($cache_metadata);
```

3. Inspection et Débogage

```
● → drupal_dev curl -I http://dev-drupal.local/api/articles
HTTP/1.1 200 OK
Date: Thu, 12 Mar 2026 18:03:43 GMT
Server: Apache/2.4.66 (Unix) PHP/8.3.30
X-Content-Type-Options: nosniff
X-Powered-By: PHP/8.3.30
Cache-Control: must-revalidate, no-cache, private
X-Drupal-Dynamic-Cache: MISS
Content-language: en
X-Frame-Options: SAMEORIGIN
Expires: Sun, 19 Nov 1978 05:00:00 GMT
X-Generator: Drupal 11 (https://www.drupal.org)
Content-Length: 142
X-Drupal-Cache: MISS
Content-Type: application/json

● → drupal_dev curl -I http://dev-drupal.local/api/articles
HTTP/1.1 200 OK
Date: Thu, 12 Mar 2026 18:03:59 GMT
Server: Apache/2.4.66 (Unix) PHP/8.3.30
X-Content-Type-Options: nosniff
X-Powered-By: PHP/8.3.30
Cache-Control: must-revalidate, no-cache, private
X-Drupal-Dynamic-Cache: MISS
Content-language: en
X-Frame-Options: SAMEORIGIN
Expires: Sun, 19 Nov 1978 05:00:00 GMT
X-Generator: Drupal 11 (https://www.drupal.org)
Content-Length: 142
X-Drupal-Cache: HIT
Content-Type: application/json
```

FIGURE 25 – Visualisation du statut de mise en cache (MISS puis HIT) via les en-têtes HTTP.

En activant le débogage du cache (`development.services.yml`), nous pouvons inspecter les tags de cache via l'en-tête HTTP `X-Drupal-Cache-Tags` et vérifier les dépendances réelles.


```
[
  {
    "nid": 10,
    "title": "Nunca Plaageeeeeee"
  },
  {
    "nid": 4,
    "title": "Iusto Letalis Luptatum"
  },
  {
    "nid": 5,
    "title": "Humo Obruo"
  },
  {
    "nid": 2,
    "title": "Immitto"
  }
]
```

FIGURE 26 – Réponse JSON de l'API personnalisée avec gestion du cache.

4. Conclusion et Q&A

Question : *Quelle est la principale différence entre max-age et cache tags ?*

Réponse : **max-age** est une expiration basée sur le temps (statique), alors que les *cache tags* permettent une invalidation basée sur les événements (dynamique). Si un contenu change, Drupal vide le cache immédiatement via les tags, peu importe le **max-age**.

JOUR 5 : MIGRATION DE DONNÉES ET IMPORT AUTOMATISÉ

La dernière étape de cette formation a été consacrée à l'importation massive de données vers Drupal via l'API Migrate. Nous avons conçu un module sur mesure pour importer des programmes académiques à partir de fichiers CSV.

1. Architecture du Module de Migration

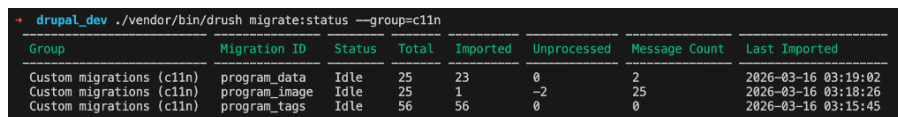
Le module personnalisé **c11n_migrate** a été structuré pour gérer trois types d'entités liés : les termes de taxonomie, les fichiers (images) et les contenus (nodes).

Composants clés :

- `migrate_plus.migration_group.c11n.yml` : Définit le groupe de migration pour une gestion commune.
- `migrate_plus.migration.program_tags.yml` : Importation des tags depuis un CSV vers le vocabulaire "Tags".
- `migrate_plus.migration.program_image.yml` : Importation des fichiers images, incluant la copie physique des fichiers vers le répertoire **public://**.
- `migrate_plus.migration.program_data.yml` : Migration principale créant les nœuds **program**, reliant les images et les tags précédemment importés.

2. Processus technique et Mapping

Le défi principal résidait dans la gestion des dépendances et de la transformation des données. Nous avons utilisé des plugins de processus avancés comme **migration_lookup** pour lier les nœuds aux images via leurs IDs respectifs générés lors de l'importation.



Group	Migration ID	Status	Total	Imported	Unprocessed	Message Count	Last Imported
Custom migrations (c11n)	program_data	Idle	25	23	0	2	2026-03-16 03:19:02
Custom migrations (c11n)	program_image	Idle	25	1	-2	25	2026-03-16 03:18:26
Custom migrations (c11n)	program_tags	Idle	56	56	0	0	2026-03-16 03:15:45

FIGURE 27 – Statut des migrations via Drush (**migrate:status**)

3. Exécution et Vérification

Les ressources ont été importées en respectant l'ordre logique des dépendances :

1. **drush migrate:import program_tags**
2. **drush migrate:import program_image**
3. **drush migrate:import program_data**

```

→ drupal_dev ./vendor/bin/drush migrate:status --group=c11n --format=json
[
  {
    "group": "Custom migrations (c11n)",
    "id": "program_data",
    "status": "Idle",
    "total": 25,
    "imported": 23,
    "unprocessed": 0,
    "message_count": 2,
    "last_imported": "2026-03-16 03:19:02"
  },
  {
    "group": "Custom migrations (c11n)",
    "id": "program_image",
    "status": "Idle",
    "total": 25,
    "imported": 1,
    "unprocessed": -2,
    "message_count": 25,
    "last_imported": "2026-03-16 03:18:26"
  },
]

```

FIGURE 28 – Détails JSON du statut de migration montrant les items importés et les messages d'erreur éventuels

4. Intégrité des données en base

Le système de migration de Drupal crée des tables de "mappage" (ex : `migrate_map_program_data`) qui permettent de garder une trace entre l'ID source du CSV et l'ID final dans Drupal. Cela permet de mettre à jour ou de supprimer les données de manière synchronisée.

source_ids_hash	sourceid1	destid1	source_row_status	role
40bd9a7953e026d7b458db71f3f40d88704d4547f114c7b819a68a94a7	learning	62	0	
2ae9b547fca82930c474a68a99ece5bfdab70a5a0b1a7da5482140e60	economics	14	0	
5fe18b4957d849493bc957a453de6c19521136c3308b937b84ee63b3b	justice	43	0	
fed5782bf8ba78fde3d1a967b06c66fc1e53960118c6a495441368763b	environment	27	0	
e24715eb5c8439505eccff135b60007d4d33395610317bc264c57a2141	health	36	0	
58f5442db0315e0a4fd8ada822ab4c846904ab5e6b6f45059311b3351	digital	19	0	
c2805725a444b0401a092d06384d087a776d31e3e13e64bb8c4756c360	public	37	0	
537ec7ce4eaf92114a369c09981a46166db582e8cfe267714068abd5ee	culture	57	0	
8bd8fae558681ca738c777bb3aea22d8b9b07914e9057f51b3d755bffc	culinary	46	0	
d7f773c8ea822cb43f115d2e4bcb5e91e413f8761e143f422c7c71294e2	psychology	32	0	
70b87fd992edc4e5905548ca92783b155ff8c554ca33c3c8a95a11d9d	management	24	0	
e9f8b3fd08542a26b6f4547d4495163f774c28df3c1ff86f22794631a8	engineering	9	0	
3f8786ea46294d3366959e82e1b6ab8da562832dvd534dea34bd70d165	global	31	0	
5dc9cf7872af4f37be915aab728bcf3a4315dd4edfedf2ae710c5adbcc	music	12	0	

FIGURE 29 – Visualisation des tables de correspondance (Mappage) en base de données

5. Questions Techniques et Bonnes Pratiques

Lors de la mise en place de ces migrations, plusieurs questions fondamentales sur le fonctionnement de l'API Migrate ont été abordées :

A. Quel est le rôle de `migration_lookup` ?

C'est le plugin essentiel pour gérer les relations. Il permet de retrouver l'identifiant (ID) d'une entité déjà importée en se basant sur son ID d'origine dans le fichier source. Par exemple, pour lier un programme à son image :

```

field_image/target_id:
  plugin: migration_lookup
  migration: program_image
  source: 'Image file'

```

B. Comment importer depuis une base MySQL au lieu d'un CSV ?

Il suffit de changer le plugin de **source** pour utiliser **sql**. On définit une requête SQL et la connexion à la base de données source :

```
source:
  plugin: sql
  query:
    - "SELECT id, title, content FROM legacy_table"
  key: migrate_db # Défini dans settings.php
```

C. Comment annuler une migration (Rollback) ?

Contrairement à une suppression manuelle, le *rollback* supprime proprement les entités créées et nettoie les tables de mappage. La commande Drush est la suivante :

```
./vendor/bin/drush migrate:rollback program_data
```

D. Comment traiter une donnée avant l'import (ex : tronquer à 10 caractères) ?

On utilise des plugins de processus pour manipuler la valeur source. Pour limiter la longueur d'une chaîne, on peut utiliser le plugin **substr** :

```
title:
  - plugin: substr
    source: Title
    length: 10
```